Building and Deploying a Web-Based Hangman Game

Individual programming assignment | October 29, 2026

Draft for student review. Confirm the reflection and be ready to explain the code before submission.

Development process

I used Codex as an AI coding agent to develop an original Hangman application incrementally. First, I supplied the assignment requirements, the larger instructor specification, the original TODO, and the instructor's Cursor transcript. The assignment requirements took precedence: only a Python/Flask server and an HTML/CSS/JavaScript browser client were needed. Terminal clients, bots, evaluations, leaderboards, and replay were excluded. The original specification and instructor TODO were preserved under docs/reference/.

The revised roadmap separated engine and word management, API, browser interface, integration testing, deployment, and submission. Work was tested and committed after each phase. The transcript guided this process; the instructor's completed source code was not copied. The instructor word lists were unavailable, so I provided five example words per difficulty. Their student-supplied provenance and small size are documented rather than presenting them as instructor data.

Implementation and request flow

The independent game.py module tracks a private word, guessed letters, incorrect letters, and a playing/won/lost status. A correct letter reveals every matching position. Six incorrect guesses cause a loss. Malformed, duplicate, and post-game guesses do not mutate the game. A safe state snapshot omits the word until the game ends. word_manager.py loads Easy, Medium, and Hard files, normalizes case, removes duplicates, validates ASCII words, and chooses randomly by difficulty.

Flask in server.py provides POST /api/games, GET /api/games/{game_id}, and POST /api/games/{game_id}/guess. Each game has a unique UUID and isolated in-memory state. Request bodies and errors are JSON. Creation returns 201; state retrieval and accepted guesses return 200. Invalid requests return 400, unknown IDs 404, post-game guesses 409, and non-JSON POSTs 415. A lock protects operations from overlapping requests in the shared process.

The browser uses semantic HTML, responsive CSS, and plain JavaScript. It offers difficulty selection, clickable letter buttons, keyboard input, a six-part SVG drawing, guess history, outcomes, and restart without reloading. A letter click or keypress calls fetch() with a letter. Flask validates it and invokes the engine, then returns JSON. JavaScript renders that server-provided snapshot. It does not select the word or decide whether a guess wins or loses the game.

Testing and debugging

The final automated suite has 29 unittest tests covering game rules, word resources, all required API routes, isolated games, full wins/losses, malformed requests, duplicate guesses, and answer disclosure. A concurrent-request test confirms that two identical simultaneous guesses spend only one life. The suite passed on Windows/Python 3.11.5, Ubuntu/Python 3.12.3, and Render/Python 3.11.18.

Live Chrome checks covered mouse and keyboard play, all difficulties, complete wins/losses, restart, responsive layouts, and server-outage/stale-game recovery. Chrome Developer Tools showed a guess payload of {"letter":"e"} and an active JSON response containing pattern, guesses, lives, and status without a word field. JavaScript updated the displayed pattern from that response. A public HTTPS smoke script separately verified the deployed page, assets, endpoints, all difficulties, completed games, validation, and fresh-game creation.

One concrete issue was keyboard focus disappearing when a clicked letter button was disabled during a request. The fix restores focus to an available letter, or to New game when the game ends. Another fix clears the old pattern if state recovery fails. Disabled-letter contrast was improved. Keyboard focus, live region semantics, drawing descriptions, and selected color contrasts were checked; screen-reader speech and full cross-browser accessibility were not.

Deployment

The repository is https://github.com/nebraassattar/web-based-hangman-game. The application is deployed at https://web-based-hangman-game.onrender.com. Git history was preserved when the local work was merged with GitHub's initial commit. Render builds from main, installs requirements, and runs the tests. Gunicorn serves server:app on Render's PORT with one worker and four threads. One worker is essential because games are stored in process memory. Gunicorn was verified under Linux rather than relying on Flask's Windows development server.

The public app was tested with the local server stopped: a complete game ended in a loss, displayed the answer, and restarted on another difficulty. Free hosting can sleep when idle, and restarts/deploys erase active games. There is no persistence or game expiry. Automatic deploys are enabled in Render, but a documentation-only push did not produce an observed follow-up deployment; the live application code is unchanged. This limitation is recorded for future updates.

Representative prompts

1. "For this first step, prepare the project documentation only. Do not implement application code yet."

2. "Okay let's start phase 1"

3. "cool let's push everything to github then continue"

Reflection - draft for review

This project illustrates why AI-assisted development still requires an explicit scope and verification. Keeping the engine independent made its rules testable before HTTP or browser code existed. Inspecting a request and response connected the visible interface to the server's decisions. Small phases, tests, documented failures, and Git commits made the generated implementation easier to inspect and explain. Before submitting, I should confirm that these points reflect my own learning and practice explaining the request flow and deployment limitations.